CEBU INSTITUTE OF TECHNOLOGY UNIVERSITY

COLLEGE OF COMPUTER STUDIES

Software Requirements Specifications
for

WILDCAT WELFARE

Change History

Date	Revision	Description
05/06/2026	1.0	Initial Draft.
09/19/2026	2.0	Post-MVP System Update: • Added Module 1.3: Animal Intake, Registry & Tag Generation (AddNewPet.jsx, 300 DPI HTML5 canvas QR collar tag generation, PetAISearchAPIView heuristic search). • Added Module 3.2: Landmark-Assisted Sighting Telemetry & 1-Click Intake (ReportSightingView.jsx with standardized CIT-U campus landmarks, emergency sentinel, and 1-Click Sighting-to-Intake handoff into AddNewPet). • Added Module 4.1: Interactive User Onboarding Walkthrough (UserGuideModal.jsx addressing SUS benchmark validation feedback). • Added Module 4.2: Role-Based Access Control & Institutional Authentication (RegisterUserView enforcing @cit.edu domain and ID format XX-XXXX-XXX, LoginView session JWT issuance, 15-minute sliding inactivity timeout, role-based dashboard routing). • Overhauled Module 2.2 to physical supply donation intake and warehouse inventory control across standardized units (kg, cans, vials, pieces). • Aligned clinical records with Republic Act No. 9482 rabies biosecurity protocols and mandatory 14-day bite observation quarantine. • Updated technical stack specifications to Django REST Framework (Python 3.12), React 19, Supabase PostgreSQL, Supabase Cloud Storage, and Render keep-alive watchdog.

Table of Contents

Change History	2
Table of Contents	3
1. Introduction	4
1.1. Purpose	4
1.2. Scope	4
1.3. Definitions, Acronyms and Abbreviations	4
1.4. References	5
2. Overall Description	6
2.1. Product perspective	6
2.2. User characteristics	6
2.4. Constraints	6
2.5. Assumptions and dependencies	7
3. Specific Requirements	8
3.1. External interface requirements	8
3.1.1. Hardware interfaces	8
3.1.2. Software interfaces	8
3.1.3. Communications interfaces	8
3.2. Functional requirements	9
Module 1	9
1.1: QR-Coded Profile Access	9
1.2: MDC Medical Dashboard	14
1.3: Animal Intake, Registry & Tag Generation	17
Module 2	20
2.1: Submit and Process Adoption Application	20
2.2: Supply Logistics & Inventory Control	25
Module 3	27
3.1: Homepage/Newsfeed	27
3.2: Landmark-Assisted Sighting Telemetry & 1-Click Intake	31
Module 4	34
4.1: Interactive User Onboarding Walkthrough	34
4.2: Role-Based Access Control & Institutional Authentication	36
3.4 Non-functional requirements	39
Performance	39
Security	39
Reliability	40
Usability	40

1. Introduction

1.1. Purpose

The purpose of the Wildcat Welfare Portal is to digitally operate the current Animal Welfare Program needed at Cebu Institute of Technology - University (CIT-U). This system is a web-based initiative to connect the program's fragmented paper record keeping into one consolidated system under one governance structure. The purpose of this document is to provide all stakeholders (Development Team, Task Force Bruno, Medical-Dental Clinic Personnel, Academic Evaluators) with an easily understandable description of the portal's complete software requirements, system features, and operational constraints.

1.2. Scope

The Wildcat Welfare Portal is a web-based platform created specifically to manage campus animal welfare and population health within the grounds of CIT-U. This system will be used by Task Force Bruno, the Medical and Dental Clinic (MDC), student volunteers, faculty, campus visitors, and prospective adopters.

The system encompasses four primary modules:

- **Module 1 (Veterinary & Animal Identity Core):** Provides public QR collar tag scanning, digital Pet Passport views, medical treatment journaling, vaccination batch tracking, and mandatory 14-day bite observation quarantine monitoring under Republic Act No. 9482, as well as baseline animal intake with 300 DPI canvas collar tag generation.
- **Module 2 (Community Engagement & Welfare Operations):** Enables prospective adopters to submit structured digital applications for eligible pets, provides administrative review workflows, and manages warehouse supply donations and inventory without monetary payment processing.
- **Module 3 (Campus Surveillance & Telemetry):** Features an aggregated community newsfeed and landmark-assisted sighting reporting (without GPS reliance) with emergency triage flags and a 1-Click handoff into official animal intake.
- **Module 4 (Institutional Security, Governance & Onboarding):** Provides an interactive 4-step onboarding guide (UserGuideModal.jsx) addressing SUS usability feedback, strict institutional authentication (@cit.edu and XX-XXXX-XXX ID format), role routing, and 15-minute sliding session inactivity termination.

1.3. Definitions, Acronyms and Abbreviations

Term/Acronym	Definition
CIT-U	Cebu Institute of Technology - University; the host academic institution.
Task Force Bruno	The official CIT-U committee responsible for the welfare, monitoring, and humane management of campus animals.
MDC	Medical-Dental Clinic; the university clinic responsible for treating campus animal bite/scratch incidents and logging veterinary treatments.
PCO	Property and Campus Operations office; involved in campus animal monitoring and facility coordination.

SSD	Student Services Division; stakeholder unit involved in campus pet welfare advocacy.
Stray	An unregistered or uncataloged animal that has entered campus premises and has not yet been formally assessed by Task Force Bruno.
RA 9482	Republic Act No. 9482 (Anti-Rabies Act of 2007); mandates pet immunization, registration, and strict 14-day observation quarantine for biting animals.
Pet Passport	A responsive public web view rendered upon scanning a pet's QR collar tag, displaying verified vaccination credentials, quarantine status, and emergency contact details.
300 DPI Canvas QR	High-resolution collar tag graphic rendered client-side on an HTML5 canvas at 300 dots per inch for direct printing onto physical waterproof collar tags.
Landmark Telemetry	Campus location reporting using predefined CIT-U physical landmarks (NGE Building, Canteen, Main Building, Library, etc.) to eliminate mobile GPS inaccuracies.
1-Click Intake	Automated handoff mechanism transferring unverified sighting data directly into the official intake form via sessionStorage.
SUS	System Usability Scale; standardized 10-item usability evaluation benchmarked at 62.32 during MVP testing and addressed via Module 4.1.
RBAC	Role-Based Access Control; separates user privileges across Community Members, MDC Clinic Staff, Shelter Staff, and Administrators.
JWT	JSON Web Token; used for authenticated API sessions, paired with a client-side 15-minute sliding inactivity watchdog.
Supabase	Managed backend platform providing relational PostgreSQL 15 database storage and S3-compatible cloud storage buckets for photos.
Render	Containerized cloud host running the Django REST Framework API, paired with a keep-alive health check to eliminate cold starts.

1.4. References

- International Organization for Standardization. (2018). ISO/IEC/IEEE 29148:2018 Systems and software engineering - Life cycle processes - Requirements engineering.
- Republic of the Philippines. (2007). Republic Act No. 9482: Anti-Rabies Act of 2007.
- Cebu Institute of Technology - University. (2024). Institutional Animal Welfare & Safety Guidelines.
- Task Force Bruno Team. (2026). Software Design Document for Wildcat Welfare Portal (SDD v2.0).
- Brooke, J. (1996). SUS: A 'quick and dirty' usability scale. Usability Evaluation in Industry, Taylor & Francis.

2. Overall Description

2.1. Product perspective

The Wildcat Welfare Portal is a web-based management system designed to replace the fragmented paper-based record-keeping currently used by CIT-U's Task Force Bruno. The system operates as a centralized digital governance layer that interfaces with the daily operations of the Medical-Dental Clinic (MDC) and campus welfare volunteers.

The system follows a modern decoupled client-server architecture:

- **Frontend:** A mobile-responsive Single-Page Application (SPA) built with React 19 and Vite. It provides touch-friendly interfaces for optical QR code scanning, public pet passports, community sighting submissions, adoption forms, and administrative dashboards.
- **Backend API:** A RESTful API service built with Django 5.x and Django REST Framework running on Python 3.12 within a Linux container on Render. It handles business logic, permission verification, medical logs, and adoption processing.
- **Database & Storage:** A managed PostgreSQL 15 database hosted on Supabase storing relational records, paired with Supabase Object Storage buckets for companion profile photos, sighting images, and medical documents.
- **Notification Service:** Automated SMTP mail transport for sending transactional status updates for adoption applications and emergency triage notifications.

2.2. User characteristics

The system serves five primary user classes with varying levels of technical expertise and access rights:

- **Task Force Admins & Welfare Officers:** The primary decision-makers who require full administrative access to animal intake, adoption vetting, supply inventory, and campus bulletins. They possess moderate to high technical proficiency.
- **MDC Clinic Staff:** Healthcare personnel responsible for logging clinical treatments, updating vaccine batch records, recording bite/scratch incidents, and overseeing the mandatory 14-day rabies observation quarantine.
- **Student Volunteers / Moderators:** Active students who assist in monitoring campus animals, verifying sightings, reporting stray animals, and managing community newsfeed updates.
- **University Community (Students/Faculty/Staff):** Students, faculty, and staff with official @cit.edu accounts who use the portal to scan collar tags, view health passports, submit sightings, and interact on the community newsfeed.
- **Potential Adopters:** Campus community members or approved public visitors who browse adoptable pets, submit digital adoption applications, and track application status.

2.4. Constraints

The development and operation of the Wildcat Welfare Portal are subject to the following constraints:

- **Connectivity:** The system depends on stable campus Wi-Fi or mobile data (4G/5G) for real-time QR code scanning, medical updates, and incident reporting.

- **Geographic Limitation & Zero GPS:** The system's operational scope is strictly limited to CIT-U campus premises. To avoid indoor GPS signal loss and high battery drain, the system relies strictly on predefined CIT-U structural landmarks (e.g., NGE Building, Canteen, Library) rather than device GPS coordinates.
- **Regulatory Compliance (RA 9482):** Handling of health and bite records must comply with university health privacy standards and Republic Act No. 9482. Any animal involved in a bite incident must be locked under a 14-day observation quarantine status.
- **Non-Monetary Policy:** In adherence to university policies regarding student projects, monetary transactions and online payment gateways are prohibited. Resource mobilization is conducted strictly through in-kind physical supply donations (kg of food, cans, medical vials, collars).
- **Institutional Domain Filtering:** Account registration requires validation of institutional email addresses (@cit.edu) and student/faculty ID format (XX-XXXX-XXX).
- **Hardware Independence:** The mobile interface must remain lightweight and responsive enough to run smoothly on standard mid-range smartphones without requiring native mobile app installation.

2.5. Assumptions and dependencies

Assumptions:

- All authorized users (Task Force Bruno personnel, student volunteers, MDC staff) possess basic digital literacy and can navigate web-based forms on desktop and mobile browsers.
- Smartphones used by students and staff for QR code scanning have functional rear cameras and modern web browsers supporting HTML5 standards.
- Task Force Bruno and MDC personnel will regularly update pet profiles, clinical notes, supply logs, and adoption statuses to ensure data reliability.
- The university administration will continue supporting the CNVR (Catch, Neuter, Vaccinate, and Return) program and campus animal welfare operations.

Dependencies:

- The system depends on cloud infrastructure availability, specifically Supabase (PostgreSQL and Object Storage) and Render (Django container hosting).
- The system depends on an automated Keep-Alive ping (targeting GET /api/health/ every 10 minutes) to prevent Render container sleep and eliminate cold-start latency for field volunteers.
- Collar QR identification depends on the physical availability and readability of 300 DPI laminated collar tags affixed to recognized university animals.
- Automated notifications depend on third-party SMTP email transport services.

3. Specific Requirements

3.1. External interface requirements

The Wildcat Welfare Portal interacts with several hardware and software components to support on-site QR scanning, clinical documentation, sighting reports, and adoption workflows.

3.1.1. Hardware interfaces

- **Smartphones and Tablets with Built-in Cameras:** Used by students, staff, and security personnel to optically scan physical collar QR codes and capture on-site sighting photos.
- **Desktop and Laptop Computers:** Used by Task Force Bruno coordinators and MDC clinic staff to access administrative dashboards, clinical records, and intake forms.
- **Printers for Collar Tags:** Standard office inkjet or laser printers used to print client-side generated 300 DPI collar tag graphics for physical lamination and collar attachment.
- **Network Devices:** Campus Wi-Fi routers and cellular base stations providing internet connectivity across CIT-U grounds.

3.1.2. Software interfaces

- **Web Browsers:** Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge on mobile and desktop platforms.
- **Database Management System:** Supabase managed PostgreSQL 15 database for relational entity storage (users, pets, medical logs, inventory, sightings, adoptions).
- **Cloud Object Storage:** Supabase Cloud Storage buckets with CDN distribution for persisting companion photos, sighting attachments, and diagnostic files.
- **Backend Web Application Framework:** Django 5.x REST Framework executing on Python 3.12, handling RESTful API requests, serialization, and role-based permissions.
- **Canvas Graphic Engine:** Client-side HTML5 canvas API for rendering 300 DPI high-density QR collar tags.
- **Email Service:** SMTP email service for dispatching automated adoption status notices and emergency alerts.

3.1.3. Communications interfaces

- **HTTP/HTTPS Protocol:** All client-server communications are conducted over HTTPS (TLS 1.3) to ensure encrypted transmission of credentials, medical data, and personal adoption records.
- **RESTful JSON APIs:** The frontend communicates with backend services via REST APIs using standard JSON request and response payloads.
- **JWT Authentication:** Secure user authentication utilizes JSON Web Tokens (JWT) with bearer authorization headers and a 15-minute sliding inactivity watchdog.
- **Keep-Alive Watchdog:** An automated cron job executes an HTTP GET request to /api/health/ every 10 minutes to maintain container responsiveness and eliminate cold-start delays.

3.2. Functional requirements

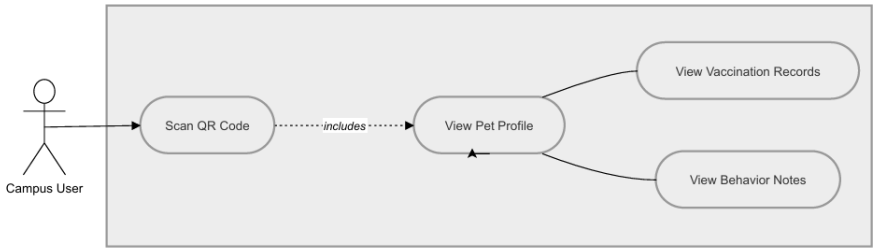
This section presents the detailed functional requirements organized across four core modules and nine functional use cases. Each use case includes its Use Case Diagram, structured Use Case Description table, Activity Diagram, and UI Wireframe.

Module 1

Veterinary & Animal Identity Core: Manages companion identification, public QR health passports, clinic health journaling, anti-rabies vaccination lot tracking, 14-day bite observation quarantine (RA 9482), and animal intake with 300 DPI canvas tag generation.

1.1: QR-Coded Profile Access

- **Use Case Diagram**

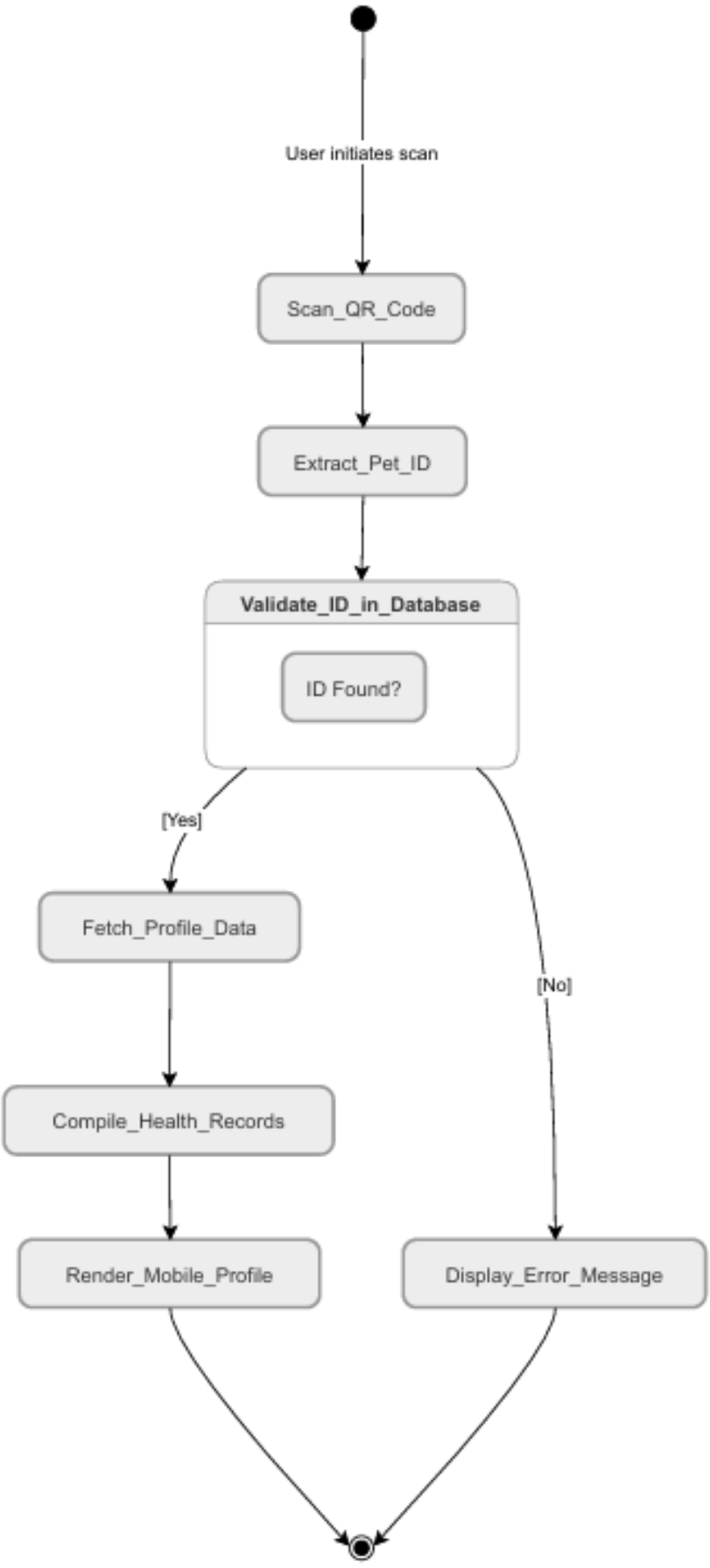


- **Use Case Description**

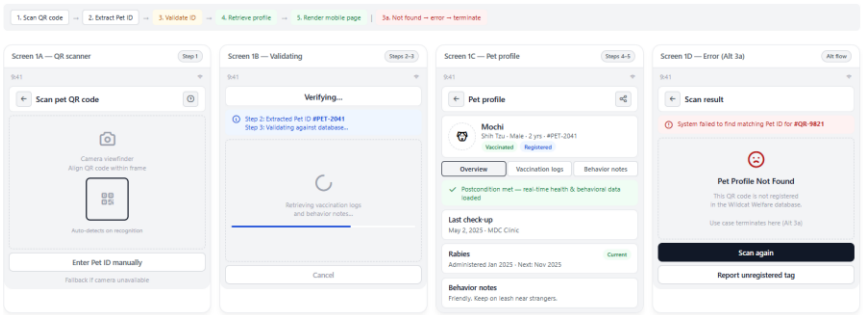
Use Case ID	UC-1.1
Use Case Name	QR-Coded Profile Access
Primary Actor	Campus User (Student, Faculty, Staff, or Visitor)
Preconditions	The user has a smartphone with camera functionality. The scanned QR code is registered in the database.
Main Success Scenario	1. The User scans the physical QR code attached to the animal's collar. 2. The mobile browser opens the URL encoded in the QR code (https://wildcatwelfare.cit.edu/pet-passport/{id}/). 3. The System validates the Pet ID against the database. 4. The System retrieves the associated pet profile (name, photo, vaccination status, territory, and behavior notes). 5. The System renders the mobile-responsive Public Pet Passport page displaying verified vaccination badges and welfare contact details.
Alternative Flows	3a. Invalid QR Code: 1. The System fails to find a matching Pet ID. 2. The System displays an error: 'Pet Profile Not Found.' 3. The System displays a link prompting the user to report an uncataloged stray (UC-3.2). 4a. Animal Under Active Bite Quarantine: 1. The animal is flagged under active 14-day bite observation. 2. The System displays a red warning banner: 'BIOSECURITY ALERT: Animal is currently under active 14-day bite surveillance. Do not approach. Contact MDC Clinic immediately.' 3. The System provides a one-tap phone dialer button to the CIT-U MDC Clinic.

Postconditions	The user successfully views the real-time health and behavioral data of the campus pet without needing to log in.

- **Activity Diagram**

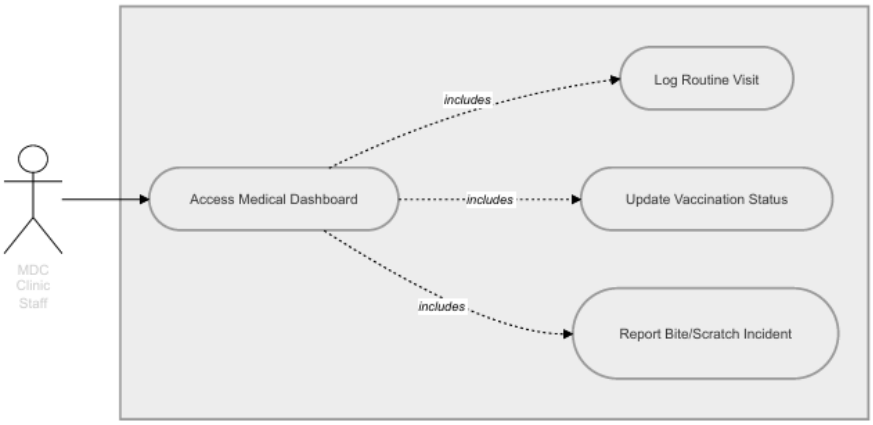

User initiates scan
Scan_QR_Code
Extract_Pet_ID
Validate_ID_in_Database
ID Found?
[Yes]
[No]
Fetch_Profile_Data
Compile_Health_Records
Render_Mobile_Profile
Display_Error_Message

- **Wireframe**


1. Scan QR code
2. Extract Pet ID
3. Validate ID
4. Retrieve profile
5. Render mobile page
3a. Not found → error → terminate
Screen 1A — QR scanner
Step 1
Scan pet QR code
Camera viewfinder
Align QR code within frame
Auto-detects on recognition
Enter Pet ID manually
Fallback if camera unavailable
Screen 1B — Validating
Steps 2–3
Verifying...
Step 2: Extracted Pet ID #PET-2041
Step 3: Validating against database...
Retrieving vaccination logs and behavior notes...
Cancel
Screen 1C — Pet profile
Steps 4–5
Pet profile
Mochi
Shih Tzu · Male · 2 yrs · #PET-2041
Vaccinated
Registered
Overview
Vaccination logs
Behavior notes
Postcondition met — real-time health & behavioral data loaded
Last check-up
May 2, 2025 · MDC Clinic
Rabies
Current
Administered Jan 2025 · Next: Nov 2025
Behavior notes
Friendly. Keep on leash near strangers.
Screen 1D — Error (Alt 3a)
Alt flow
Scan result
System failed to find matching Pet ID for #QR-9821
Pet Profile Not Found
This QR code is not registered in the Wildcat Welfare database.
Use case terminates here (Alt 3a)
Scan again
Report unregistered tag

1.2: MDC Medical Dashboard

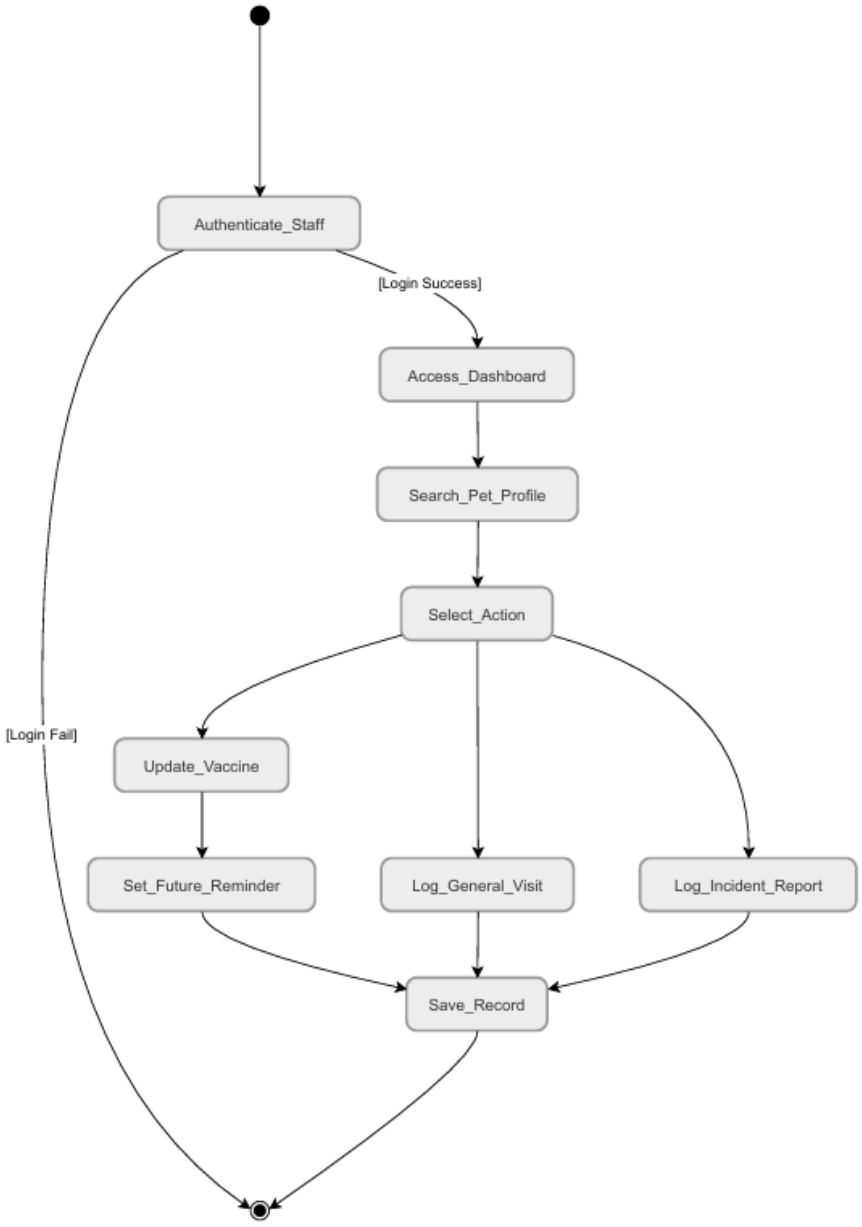
- **Use Case Diagram**



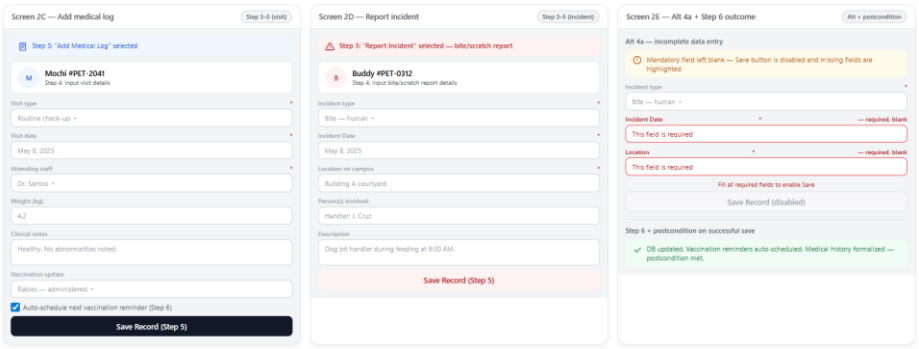
- **Use Case Description**

Use Case ID	UC-1.2
Use Case Name	Log Medical Records and Incidents
Primary Actor	MDC Clinic Staff
Preconditions	The staff member is authenticated with CLINIC role credentials and logged into the restricted medical dashboard.
Main Success Scenario	1. The Staff searches for a specific pet using the Pet ID, Name, or collar scan. 2. The System displays the active medical profile and treatment timeline. 3. The Staff selects 'Add Clinical Record' or 'Report Incident'. 4. The Staff inputs visit details: Treatment Type, Diagnosis, Medication, Vaccine Manufacturer, Batch/Lot Number, and Expiration Date. 5. The Staff clicks 'Save Record'. 6. The System updates the database and automatically calculates the next vaccination due date. 7. The pet's Public Pet Passport badge updates to 'Vaccinated - Up to Date'.
Alternative Flows	4a. Bite/Scratch Incident Logging (RA 9482 Compliance): 1. Staff selects 'Bite Incident' as the visit category. 2. The System automatically flags the animal's quarantine_status as 'UNDER_OBSERVATION'. 3. The System sets and locks a 14-day observation countdown clock. 4. The System records the bite victim's institutional ID and clinic referral details. 5. An alert is sent to Task Force Bruno personnel for physical animal monitoring. 4b. Incomplete Mandatory Fields: 1. Staff leaves required clinical fields (e.g., vaccine lot number or treatment date) blank. 2. The System disables the 'Save' button and highlights the required fields in red.
Postconditions	The animal's medical history is updated, and the campus clinic has a formalized, legal record of the treatment or incident.

- **Activity Diagram**

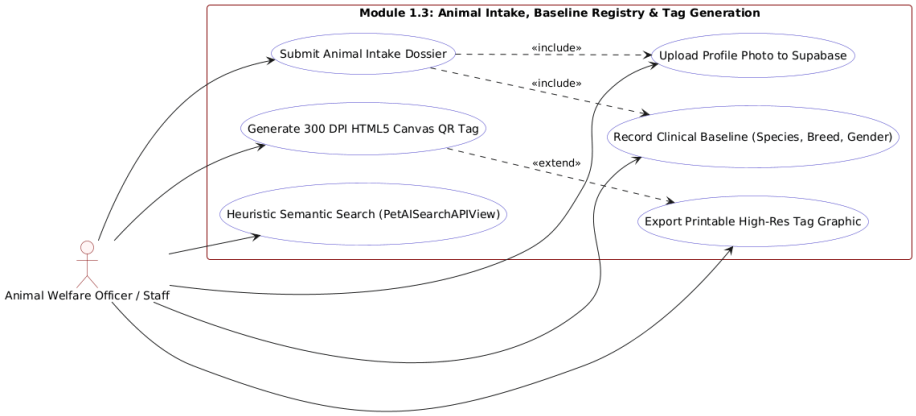


- **Wireframe**


Screen 2C — Add medical log
Step 3–5 (visit)
Step 3: "Add Medical Log" selected
Mochi #PET-2041
Step 4: Input visit details
Visit type
Routine check-up
Visit date
May 8, 2025
Attending staff
Dr. Santos
Weight (kg)
4.2
Clinical notes
Healthy. No abnormalities noted.
Vaccination update
Rabies — administered
Auto-schedule next vaccination reminder (Step 6)
Save Record (Step 5)
Screen 2D — Report incident
Step 3–5 (incident)
Step 3: "Report Incident" selected — bite/scratch report
Buddy #PET-0312
Step 4: Input bite/scratch report details
Incident type
Bite — human
Incident Date
May 8, 2025
Location on campus
Building A courtyard
Person(s) involved
Handler: J. Cruz
Description
Dog bit handler during feeding at 8:30 AM.
Save Record (Step 5)
Screen 2E — Alt 4a + Step 6 outcome
Alt + postcondition
Alt 4a — incomplete data entry
Mandatory field left blank — Save button is disabled and missing fields are highlighted
Incident type
Bite — human
Incident Date
— required, blank
This field is required
Location
— required, blank
This field is required
Fill all required fields to enable Save
Save Record (disabled)
Step 6 + postcondition on successful save
DB updated. Vaccination reminders auto-scheduled. Medical history formalized — postcondition met.

1.3: Animal Intake, Registry & Tag Generation

- **Use Case Diagram**

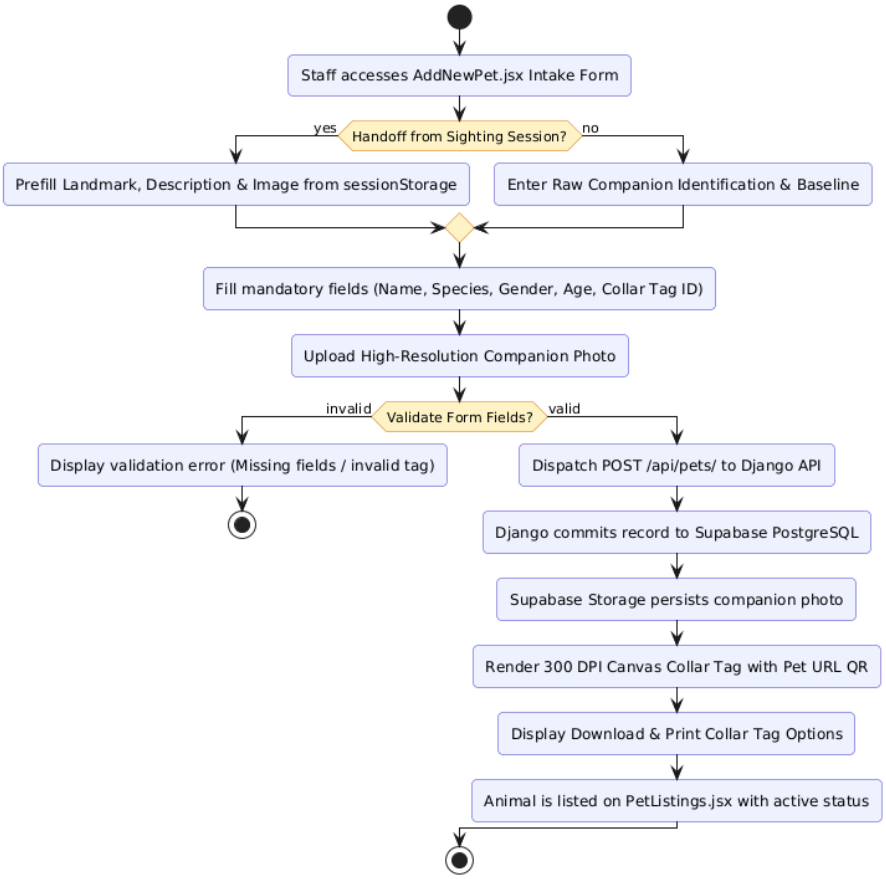


- **Use Case Description**

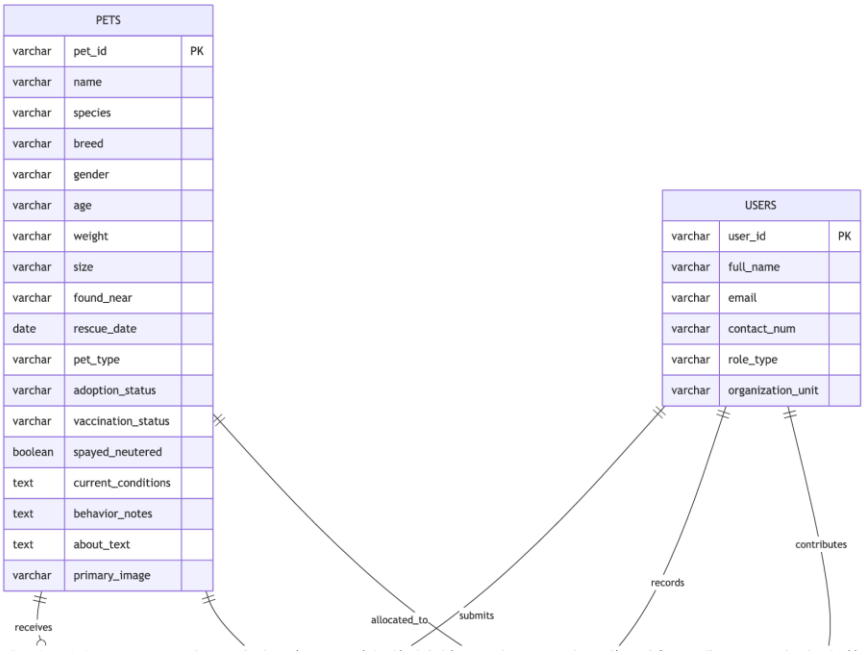
Use Case ID	UC-1.3
Use Case Name	Animal Intake, Baseline Registry & Tag Generation
Primary Actor	Task Force Bruno Staff / Welfare Officer
Preconditions	Staff is authenticated with STAFF or ADMIN credentials and accesses the AddNewPet.jsx form.
Main Success Scenario	1. Staff opens the Animal Intake form. 2. If converted from a community sighting, fields (landmark, description, photo) are automatically populated from sessionStorage. 3. Staff enters companion identity details: Name, Species (Dog/Cat), Biological Sex, Estimated Age, Color/Markings, Temperament, and Campus Territory. 4. Staff uploads a clear companion photo. 5. Staff clicks 'Register Companion & Generate Tag'. 6. The System validates the form and creates the pet record in Supabase PostgreSQL. 7. The companion photo is uploaded to Supabase Storage and associated with the record. 8. The client-side HTML5 canvas dynamically generates a 300 DPI print-ready collar tag containing the pet name, UUID, and high-density QR code. 9. Staff clicks 'Export Printable Tag' to save the graphic for physical collar production. 10. The companion is officially listed on PetListings.jsx and indexed in the search catalog.
Alternative Flows	3a. Duplicate Record Detected: 1. Heuristic search identifies an existing pet with matching physical traits and territory. 2. The System displays a warning modal: 'Possible duplicate pet detected.' 3. Staff can verify and merge with existing record or proceed with new registration. 4a. Image Upload Failure: 1. Network timeout occurs during image upload. 2. The System keeps all text fields intact and prompts staff to re-select the

	image.
Postconditions	The animal is permanently registered in the campus database, fitted with a generated 300 DPI QR collar tag, and made searchable across the portal.

- **Activity Diagram**



- **Wireframe**

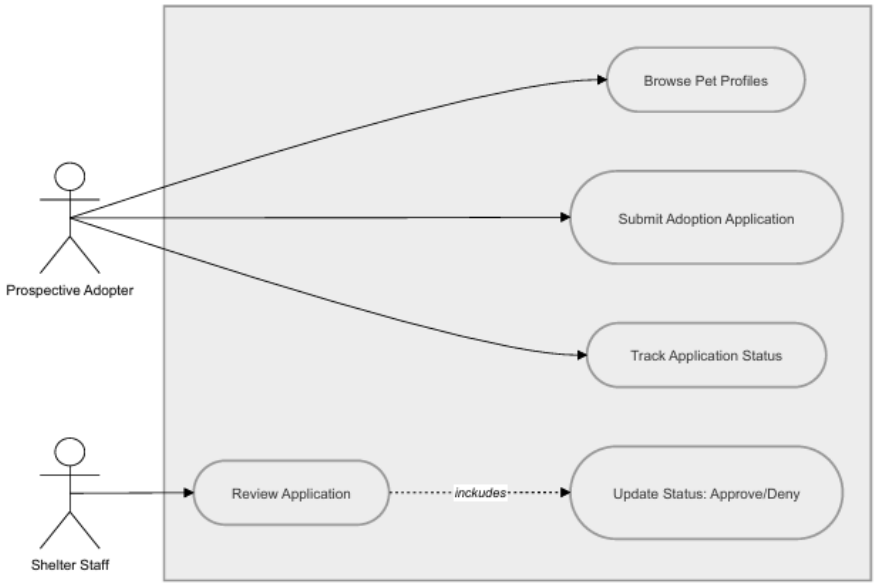

PETS
varchar pet_id PK
varchar name
varchar species
varchar breed
varchar gender
varchar age
varchar weight
varchar size
varchar found_near
date rescue_date
varchar pet_type
varchar adoption_status
varchar vaccination_status
boolean spayed_neutered
text current_conditions
text behavior_notes
text about_text
varchar primary_image
USERS
varchar user_id PK
varchar full_name
varchar email
varchar contact_num
varchar role_type
varchar organization_unit
receives
allocated_to
submits
records
contributes

Module 2

Community Engagement & Welfare Operations: Manages digital adoption applications, multi-stage applicant vetting, shelter triage workflows, and warehouse supply logistics with physical donation tracking across standardized units (kg, cans, vials, pieces).

2.1: Submit and Process Adoption Application

- **Use Case Diagram**

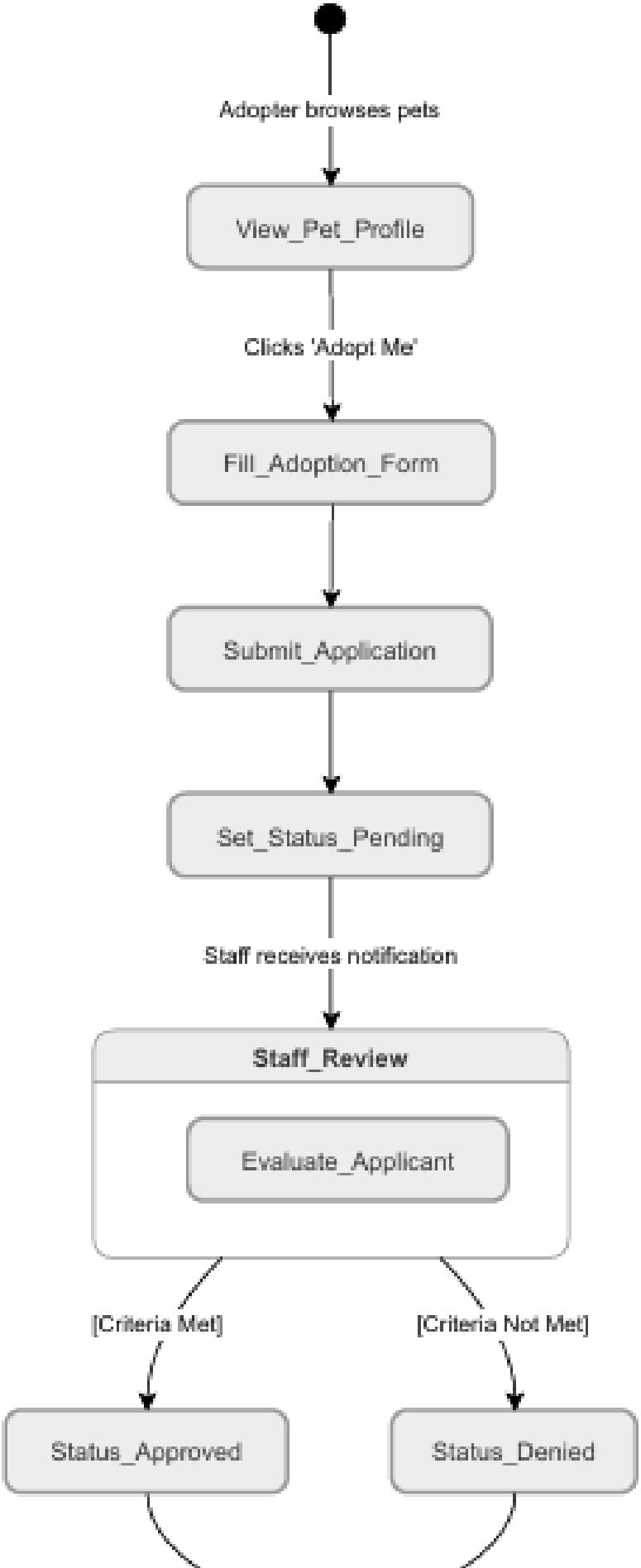


- **Use Case Description**

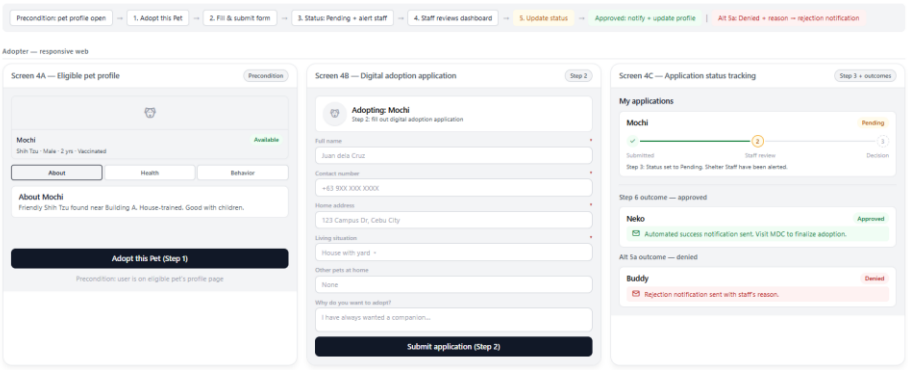
Use Case ID	UC-2.1
Use Case Name	Submit and Process Adoption Application
Primary Actor	Prospective Adopter, Shelter Staff
Preconditions	The Adopter is viewing an eligible pet's profile page on PetListings.jsx; the pet has status 'AVAILABLE_FOR_ADOPTION'.
Main Success Scenario	1. The Adopter clicks 'Apply for Adoption'. 2. The Adopter completes the digital adoption application form (housing type, pet care experience, landlord policy, daily availability, financial readiness pledge). 3. The Adopter submits the application. 4. The System sets the application status to 'Pending' and alerts Shelter Staff. 5. Staff accesses PendingApplications.jsx and reviews the application dossier. 6. Staff moves the application through triage stages: Under Review -> Interview Scheduled -> Approved. 7. The System dispatches automated email updates to the applicant upon each status change. 8. Upon final approval, the pet's profile status is updated to 'Adopted', and an automated celebratory bulletin is posted to the Community Newsfeed.
Alternative Flows	2a. Incomplete Vetting Information: 1. Adopter skips required fields in the questionnaire.

	2. The System highlights incomplete questions and prevents submission. 6a. Application Denied: 1. Staff determines housing or care conditions are unsuitable. 2. Staff sets status to 'Denied' and enters an explanatory reason. 3. The System sends a polite notification to the Adopter, and the pet remains available for other applicants.
Postconditions	The adoption application is definitively resolved, recorded in the audit trail, and the animal's ownership status is updated.

- **Activity Diagram**

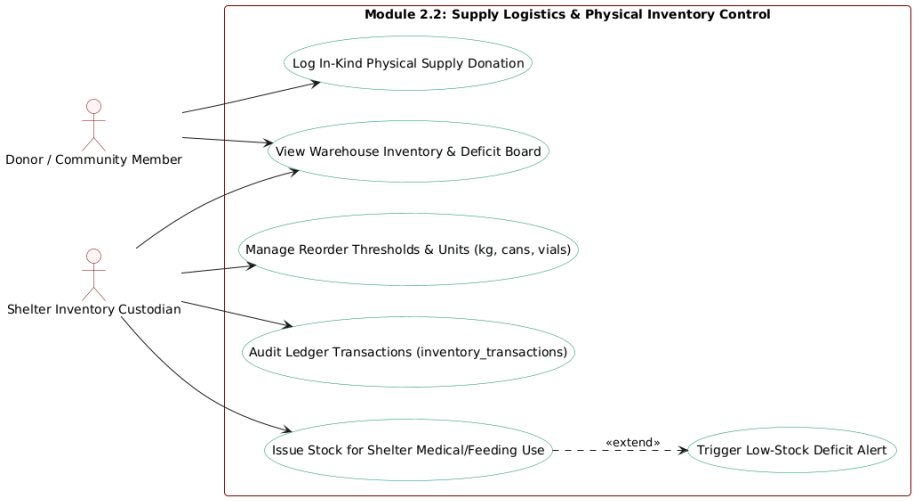

Adopter browses pets
View_Pet_Profile
Clicks 'Adopt Me'
Fill_Adoption_Form
Submit_Application
Set_Status_Pending
Staff receives notification
Staff_Review
Evaluate_Applicant
[Criteria Met]
[Criteria Not Met]
Status_Approved
Status_Denied

- **Wireframe**


Precondition: pet profile open
1. Adopt this Pet
2. Fill & submit form
3. Status: Pending + alert staff
4. Staff reviews dashboard
5. Update status
Approved: notify + update profile
Alt 5a: Denied + reason → rejection notification
Adopter — responsive web
Screen 4A — Eligible pet profile
Precondition
Mochi
Available
Shih Tzu · Male · 2 yrs · Vaccinated
About
Health
Behavior
About Mochi
Friendly Shih Tzu found near Building A. House-trained. Good with children.
Adopt this Pet (Step 1)
Precondition: user is on eligible pet's profile page
Screen 4B — Digital adoption application
Step 2
Adopting: Mochi
Step 2: fill out digital adoption application
Full name
Juan dela Cruz
Contact number
+63 9XX XXX XXXX
Home address
123 Campus Dr, Cebu City
Living situation
House with yard
Other pets at home
None
Why do you want to adopt?
I have always wanted a companion...
Submit application (Step 2)
Screen 4C — Application status tracking
Step 3 + outcomes
My applications
Mochi
Pending
Submitted
Staff review
Decision
Step 3: Status set to Pending. Shelter Staff have been alerted.
Step 6 outcome — approved
Neko
Approved
Automated success notification sent. Visit MDC to finalize adoption.
Alt 5a outcome — denied
Buddy
Denied
Rejection notification sent with staff's reason.

2.2: Supply Logistics & Inventory Control

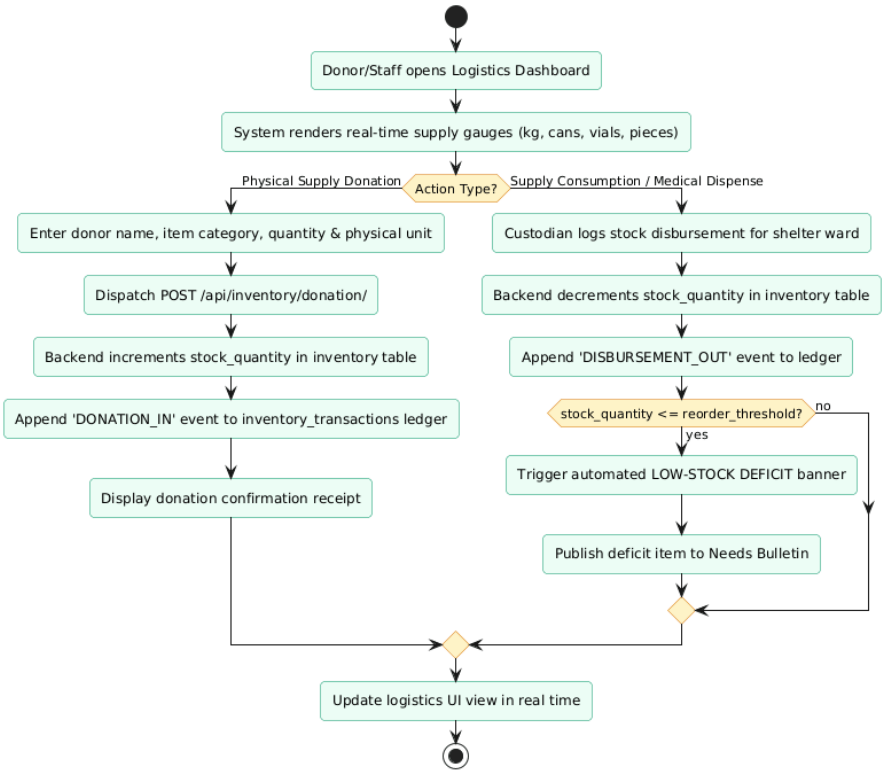
- **Use Case Diagram**



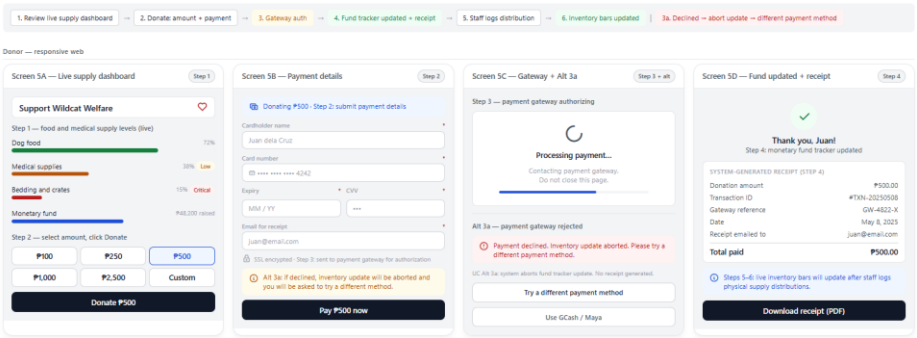
- **Use Case Description**

Use Case ID	UC-2.2
Use Case Name	Supply Logistics & Inventory Control
Primary Actor	Supply Donor, Shelter Inventory Custodian
Preconditions	The Donor accesses the public logistics dashboard; Staff has administrative access to the inventory ledger.
Main Success Scenario (In-Kind Physical Donation):	1. The Donor brings physical supplies (dog/cat food, vaccines, medicines, leashes) to the designated campus drop-off point. 2. Staff accesses the Logistics portal and clicks 'Log Physical Donation'. 3. Staff inputs Donor Name, Item Category, Quantity, Unit of Measure (kg, cans, vials, pieces), and Expiration Date. 4. Staff submits the entry. 5. The System atomically increments stock_quantity in the inventory table and appends a 'DONATION_IN' event to inventory_transactions. 6. The live inventory dashboard updates immediately, displaying current stock levels and issuing a digital acknowledgement receipt.
Alternative Flows (Stock Disbursement & Low-Stock Alerts):	2a. Staff Logs Supply Consumption: 1. Staff logs supply usage for shelter animals (e.g., 5 kg of dry cat food used). 2. The System decrements stock_quantity and appends a 'DISBURSEMENT_OUT' record to the ledger. 3. If stock_quantity falls at or below reorder_threshold, the System triggers an automated 'LOW STOCK' alert banner on the public Logistics Dashboard and publishes an urgent needs bulletin to the Community Newsfeed.
Postconditions	Warehouse inventory balances are updated in real time, full transaction history is preserved, and supply deficits are communicated to the campus community.

- **Activity Diagram**



- **Wireframe**

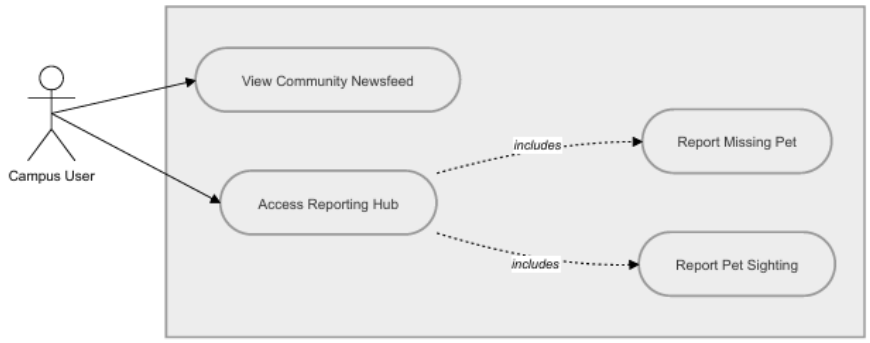


Module 3

Campus Surveillance & Telemetry: Powers the public campus newsfeed and provides landmark-based sighting telemetry for lost or stray animals, complete with emergency alerts and 1-Click conversion into official shelter intake.

3.1: Homepage/Newsfeed

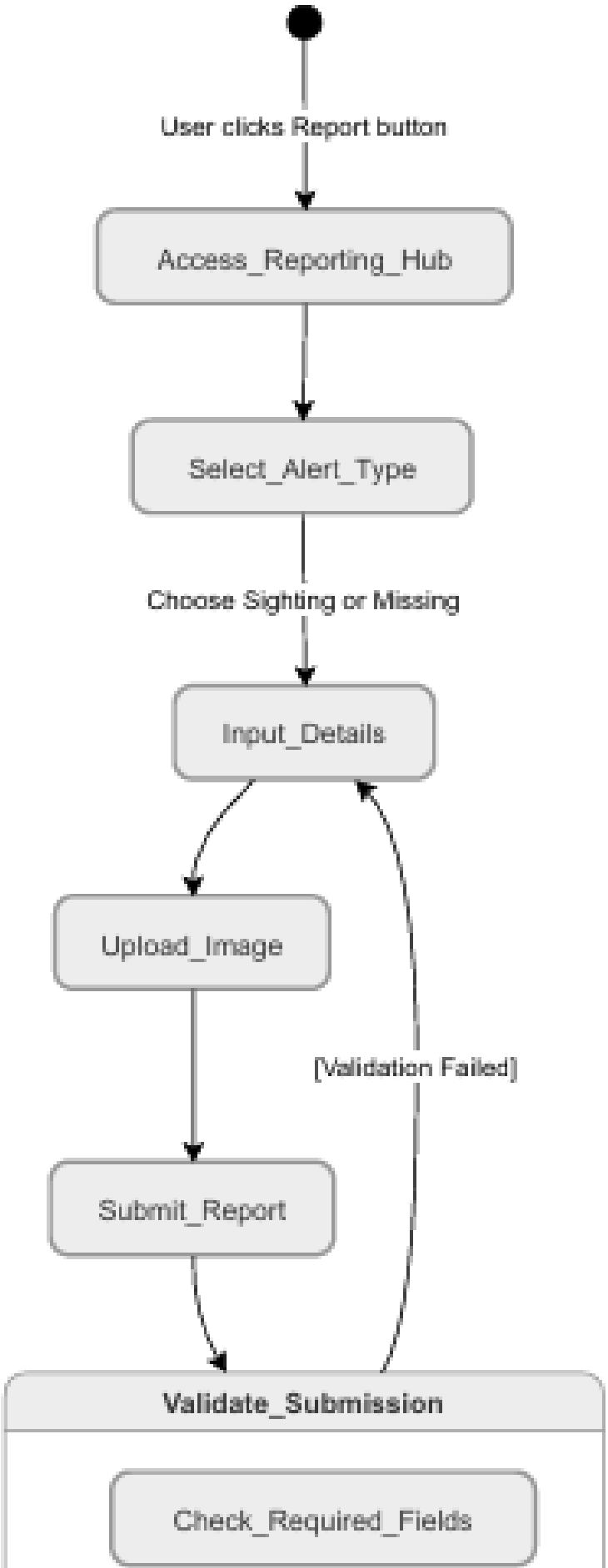
- **Use Case Diagram**



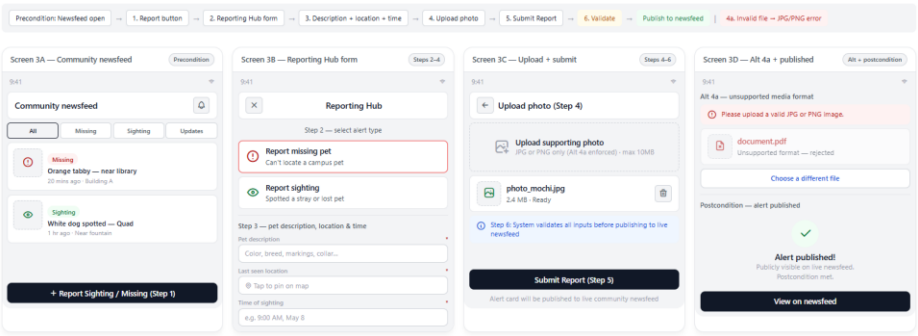
- **Use Case Description**

Use Case ID	UC-3.1
Use Case Name	Community Newsfeed and Reporting Hub
Primary Actor	Community Member (Student, Faculty, Staff, Guest)
Preconditions	The user accesses the portal homepage or Community Newsfeed tab.
Main Success Scenario	1. The User visits the homepage. 2. The System requests the unified newsfeed endpoint (/api/newsfeed/unified/). 3. The System retrieves and chronologically displays aggregated posts: official bulletins, adoption stories, emergency alerts, and community reports. 4. Pinned administrative announcements are prominently displayed at the top of the feed. 5. Authenticated users can like posts and leave threaded comments. 6. Interactive counters (likes, comment count) update immediately in real time.
Alternative Flows	5a. Unauthenticated User Interacts: 1. Guest user attempts to like or comment. 2. The System displays a sign-in modal prompting the user to log in with their CIT-U credentials.
Postconditions	The user stays informed regarding campus animal welfare news, and community interaction is logged for moderation.

- **Activity Diagram**

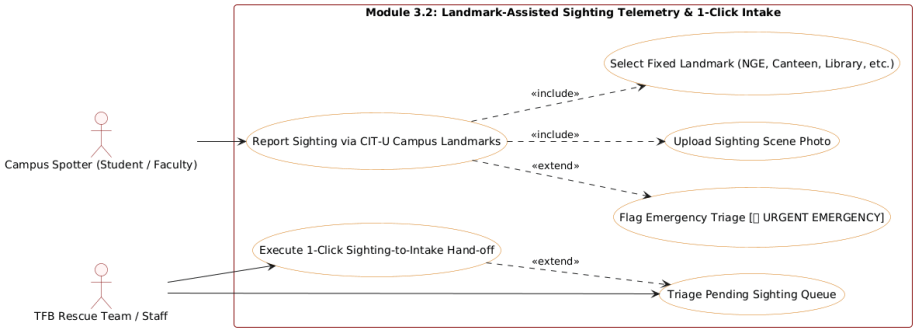

User clicks Report button
Access_Reporting_Hub
Select_Alert_Type
Choose Sighting or Missing
Input_Details
Upload_Image
[Validation Failed]
Submit_Report
Validate_Submission
Check_Required_Fields

- **Wireframe**


Precondition: Newsfeed open
1. Report button
2. Reporting Hub form
3. Description + location + time
4. Upload photo
5. Submit Report
6. Validate
Publish to newsfeed
4a. Invalid file → JPG/PNG error
Screen 3A — Community newsfeed
Precondition
Community newsfeed
All
Missing
Sighting
Updates
Missing
Orange tabby — near library
20 mins ago · Building A
Sighting
White dog spotted — Quad
1 hr ago · Near fountain
+ Report Sighting / Missing (Step 1)
Screen 3B — Reporting Hub form
Steps 2–4
Reporting Hub
Step 2 — select alert type
Report missing pet
Can't locate a campus pet
Report sighting
Spotted a stray or lost pet
Step 3 — pet description, location & time
Pet description
Color, breed, markings, collar...
Last seen location
Tap to pin on map
Time of sighting
e.g. 9:00 AM, May 8
Screen 3C — Upload + submit
Steps 4–6
Upload photo (Step 4)
Upload supporting photo
JPG or PNG only (Alt 4a enforced) · max 10MB
photo_mochi.jpg
2.4 MB · Ready
Step 6: System validates all inputs before publishing to live newsfeed
Submit Report (Step 5)
Alert card will be published to live community newsfeed
Screen 3D — Alt 4a + published
Alt + postcondition
Alt 4a — unsupported media format
Please upload a valid JPG or PNG image.
document.pdf
Unsupported format — rejected
Choose a different file
Postcondition — alert published
Alert published!
Publicly visible on live newsfeed.
Postcondition met.
View on newsfeed

3.2: Landmark-Assisted Sighting Telemetry & 1-Click Intake

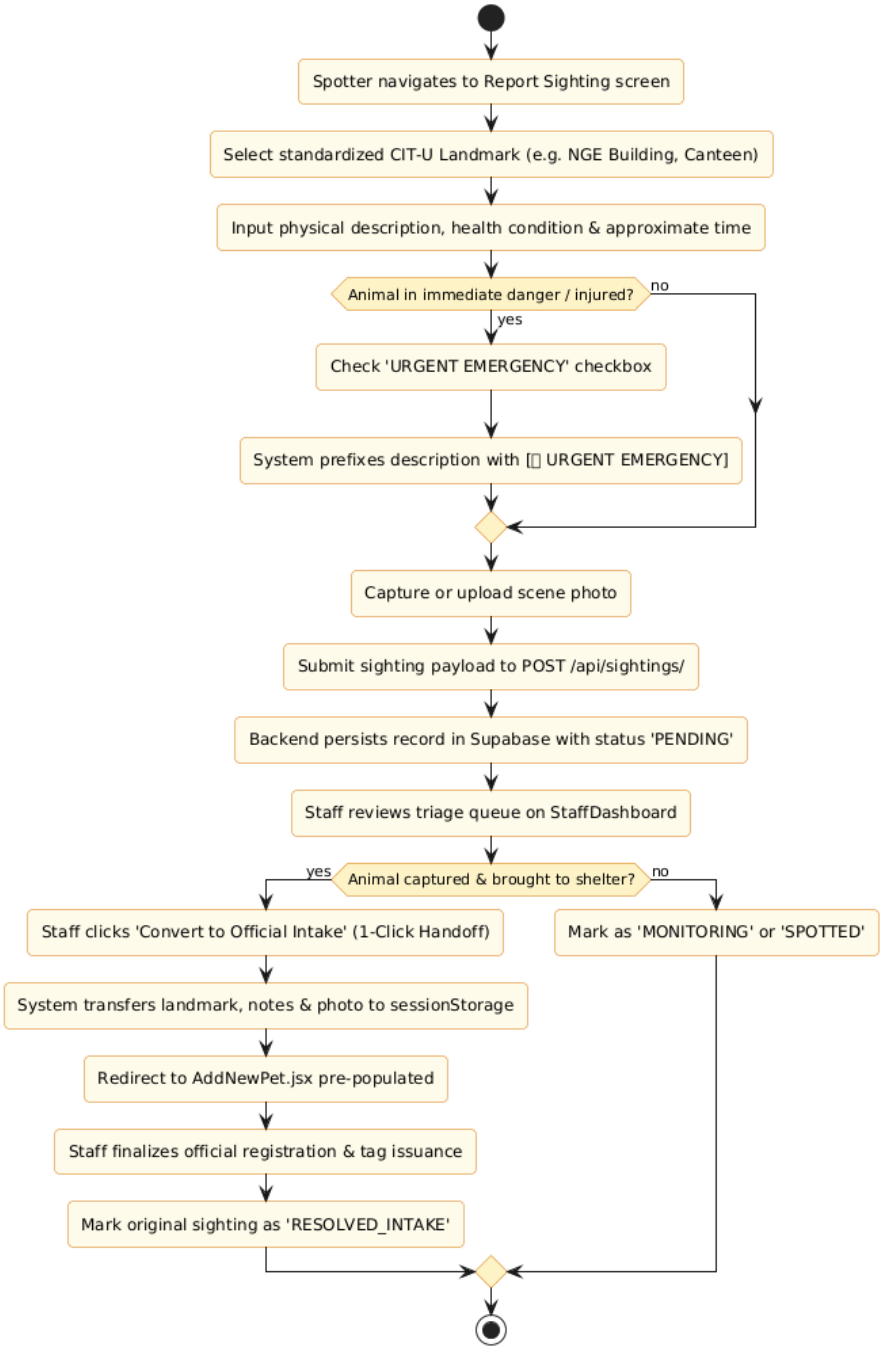
- **Use Case Diagram**



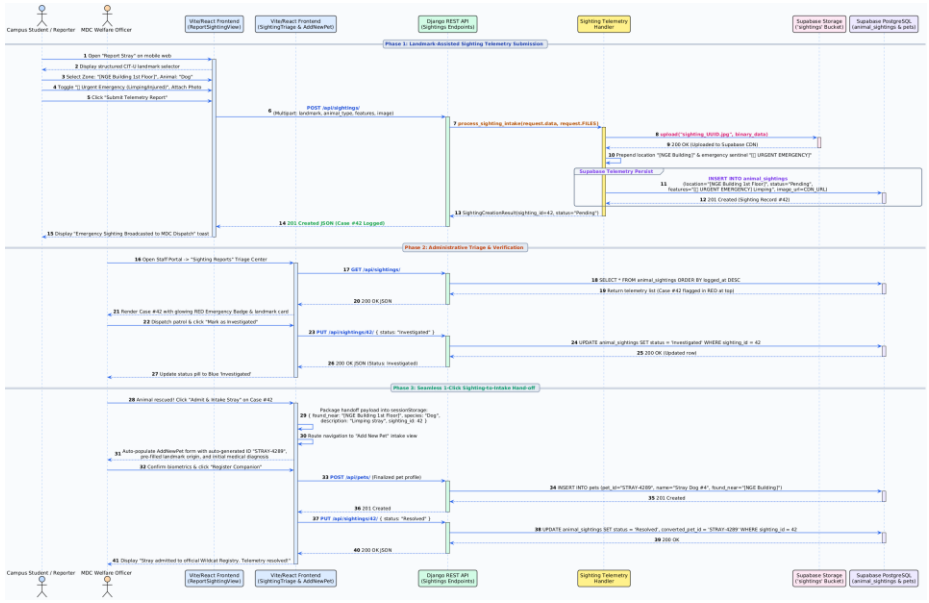
- **Use Case Description**

Use Case ID	UC-3.2
Use Case Name	Landmark-Assisted Sighting Telemetry & 1-Click Intake
Primary Actor	Campus Spotter (Student, Faculty, Security Guard), Shelter Staff
Preconditions	The spotter encounters an uncataloged, lost, or injured animal on CIT-U campus grounds.
Main Success Scenario	1. The User clicks the 'Report Sighting' button. 2. The System presents the ReportSightingView.jsx form. 3. The User selects a standardized CIT-U Campus Landmark from the dropdown: NGE Building, Canteen, Main Building, Library, SAL Building, Academic Complex, Gate 1, Gate 2, Gate 3, or Gate 4. 4. The User inputs physical description, behavior, and approximate time seen. 5. If the animal is injured or in immediate distress, the User checks 'URGENT EMERGENCY', prepending '[URGENT EMERGENCY]' to the record. 6. The User uploads a photo and clicks 'Submit Report'. 7. The System validates inputs and publishes the sighting to the staff triage dashboard. 8. When rescue volunteers secure the animal, Staff clicks 'Convert to Official Intake' (1-Click Handoff). 9. The System automatically transfers landmark, notes, and photo into sessionStorage and opens AddNewPet.jsx pre-filled. 10. Staff finalizes the baseline record, registers the companion, and issues a 300 DPI QR collar tag.
Alternative Flows	5a. Urgent Emergency Flagged: 1. System detects emergency flag and sends high-priority notification to on-duty welfare staff for immediate rescue. 6a. Unsupported Image Format: 1. The User attempts to upload an unsupported file type. 2. The System displays an error: 'Please upload a valid JPG or PNG image.'
Postconditions	The sighting report is recorded, rescue teams are alerted, and the animal is smoothly transitioned into official intake without duplicate data entry.

- **Activity Diagram**



- **Wireframe**

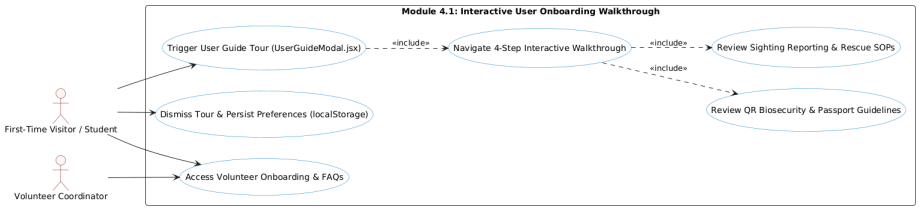

Campus Student / Reporter
MDC Welfare Officer
Vite/React Frontend (ReportSightingView)
Vite/React Frontend (SightingTriage & AddNewPet)
Django REST API (Sightings Endpoints)
Sighting Telemetry Handler
Supabase Storage ('sightings' Bucket)
Supabase PostgreSQL (animal_sightings & pets)
Phase 1: Landmark-Assisted Sighting Telemetry Submission
1 Open "Report Stray" on mobile web
2 Display structured CIT-U landmark selector
3 Select Zone: "INGE Building 1st Floor?", Animal: "Dog"
4 Toggle "[Urgent Emergency (Limping/Injured)", Attach Photo
5 Click "Submit Telemetry Report"
6 POST /api/sightings/
(Multipart: landmark, animal_type, features, image)
7 process_sighting_intake(request.data, request.FILES)
8 upload("sighting_0049.jpg", binary_data)
9 200 OK (Uploaded to Supabase CDN)
10 Prepend location "(INGE Building)" & emergency sentinel "[] URGENT EMERGENCY]"
Supabase Telemetry Persist
11 INSERT INTO animal_sightings
(location="(INGE Building 1st Floor)", status="Pending",
features="[] URGENT EMERGENCY] Limping", image_url="CDN_URL)
12 201 Created (Sighting Record #42)
13 SightingCreationResult(sighting_id=42, status="Pending")
14 201 Created JSON (Case #42 Logged)
15 Display "Emergency Sighting Broadcasted to MDC Dispatch!" toast
Phase 2: Administrative Triage & Verification
16 Open Staff Portal -> "Sighting Reports" Triage Center
17 GET /api/sightings/
18 SELECT * FROM animal_sightings ORDER BY logged_at DESC
19 Return telemetry list (Case #42 flagged in RED at top)
20 200 OK JSON
21 Render Case #42 with glowing RED Emergency Badge & landmark card
22 Dispatch patrol & click "Mark as Investigated"
23 PUT /api/sightings/42/ { status: "Investigated" }
24 UPDATE animal_sightings SET status = 'Investigated' WHERE sighting_id = 42
25 200 OK (Updated row)
26 200 OK JSON (Status: Investigated)
27 Update status pill to Blue 'Investigated'
Phase 3: Seamless 1-Click Sighting-to-Intake Hand-off
28 Animal rescued! Click "Admit & Intake Stray" on Case #42
29 Package handoff payload into sessionStorage:
{ found_near: "(INGE Building 1st Floor)", species: "Dog",
description: "Limping Stray", sighting_id: 42 }
30 Route navigation to "Add New Pet" intake view
31 Auto-populate AddNewPet form with auto-generated ID "STRAY-4289",
pre-filled landmark origin, and initial medical diagnosis
32 Confirm biometrics & click "Register Companion"
33 POST /api/pets/ (Finalized pet profile)
34 INSERT INTO pets (pet_id='STRAY-4289', name='Stray Dog #4', found_near='(INGE Building)')
35 201 Created
36 201 Created
37 PUT /api/sightings/42/ { status: "Resolved" }
38 UPDATE animal_sightings SET status = 'Resolved', converted_pet_id = 'STRAY-4289' WHERE sighting_id = 42
39 200 OK
40 200 OK JSON
41 Display "Stray admitted to official Wildcat Registry. Telemetry resolved!"

Module 4

Institutional Security, Governance & Onboarding: Handles first-time user guidance, institutional identity validation (@cit.edu), role-based access control, and sliding 15-minute inactivity session termination.

4.1: Interactive User Onboarding Walkthrough

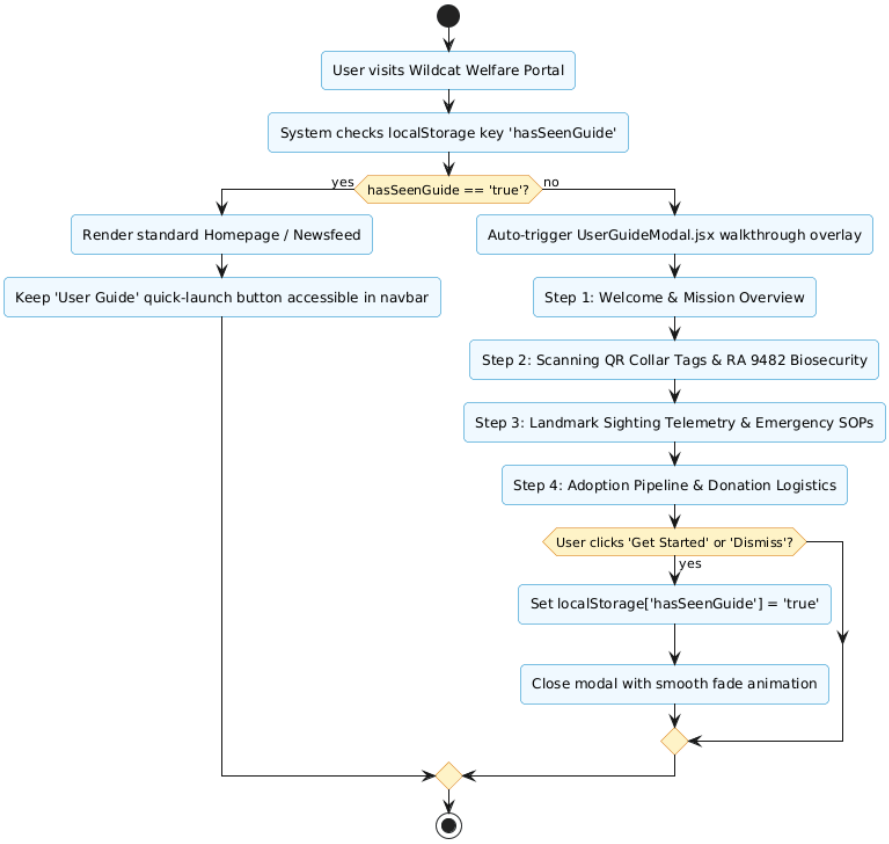
- **Use Case Diagram**



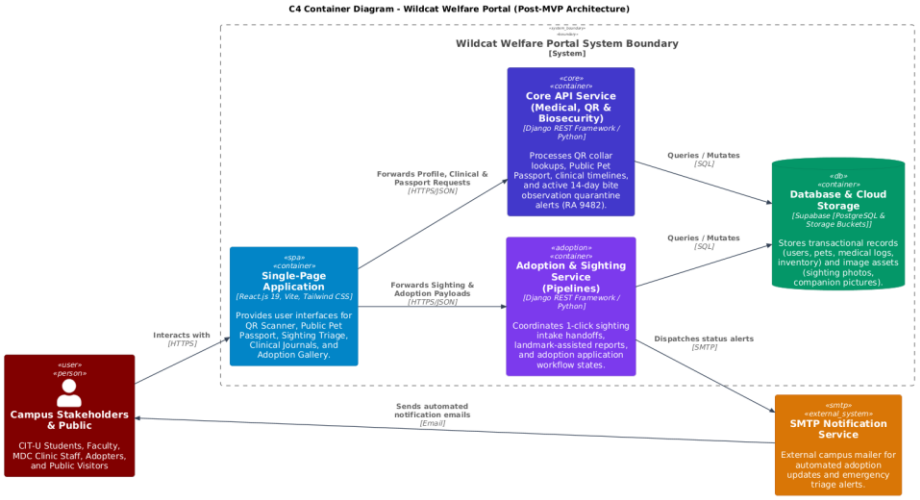
- **Use Case Description**

Use Case ID	UC-4.1
Use Case Name	Interactive User Onboarding Walkthrough
Primary Actor	First-Time Visitor, Student Volunteer, Campus User
Preconditions	The user accesses the Wildcat Welfare Portal on desktop or mobile.
Main Success Scenario	1. The System checks localStorage for the key 'hasSeenGuide'. 2. If not previously viewed, the System displays the interactive UserGuideModal.jsx walkthrough overlay. 3. Step 1 (Welcome & Mission): Explains the purpose of Task Force Bruno and campus animal welfare. 4. Step 2 (QR Collar Tagging): Demonstrates how scanning collar tags opens the public Pet Passport with verified vaccination records. 5. Step 3 (Sighting Reporting): Explains landmark-based location selection and emergency reporting protocols. 6. Step 4 (Adoption & Logistics): Outlines the adoption process and how to contribute in-kind supply donations. 7. The User clicks 'Get Started' to finish the tour. 8. The System saves the preference in localStorage and closes the modal. 9. The guide remains accessible anytime via the 'User Guide' link in the top navigation bar.
Alternative Flows	7a. User Closes Tour Early: 1. User clicks the 'Close / Skip' button at any step. 2. The modal closes immediately, saving the preference to prevent intrusive re-triggering.
Postconditions	The user understands primary portal workflows, reducing onboarding confusion and improving usability (addressing the initial 62.32 SUS validation score).

- **Activity Diagram**

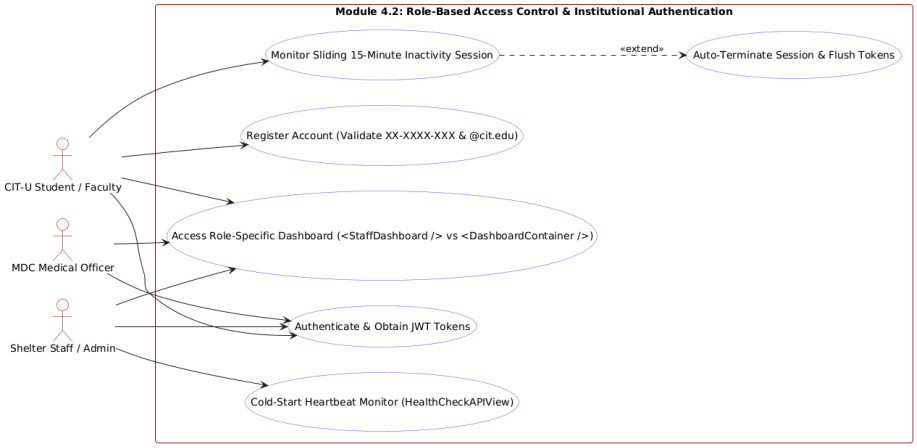


- **Wireframe**



4.2: Role-Based Access Control & Institutional Authentication

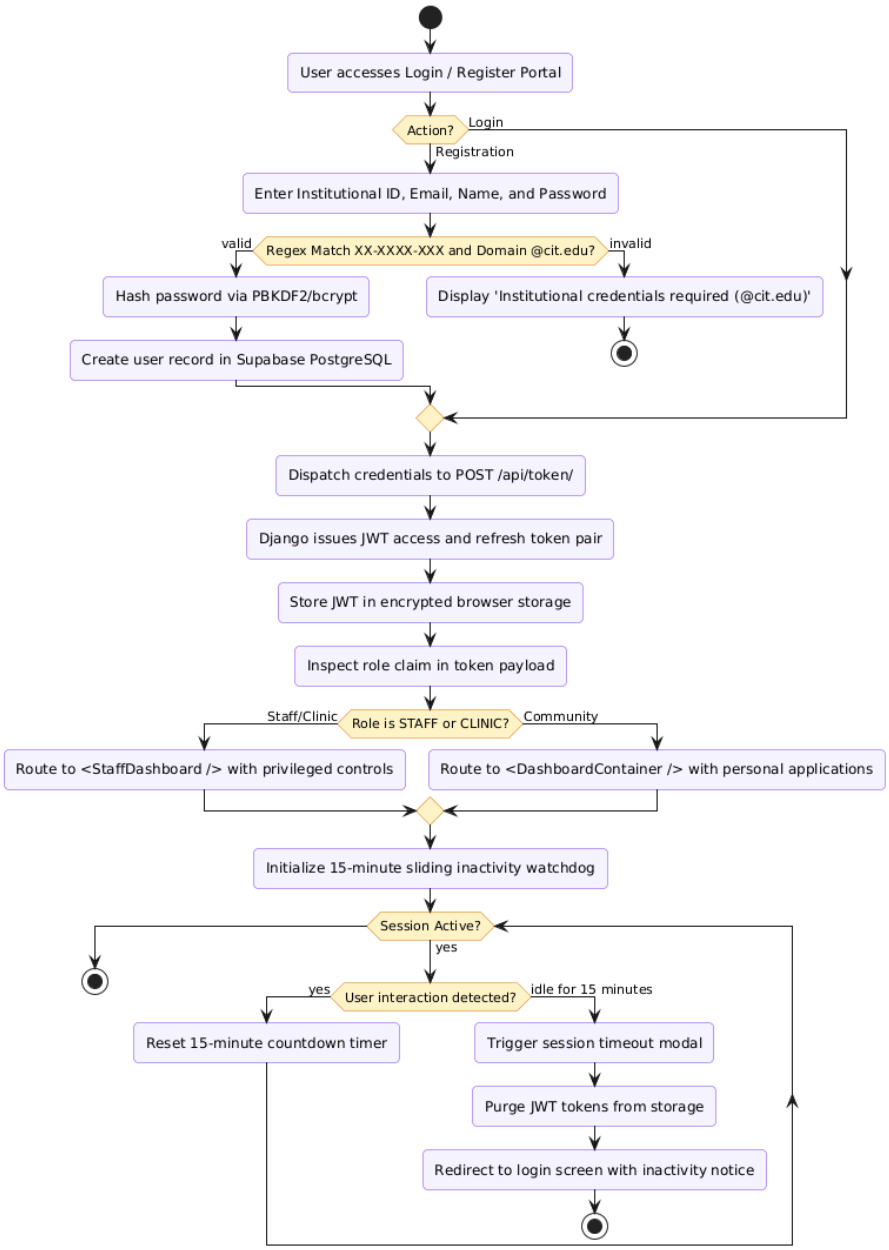
- **Use Case Diagram**



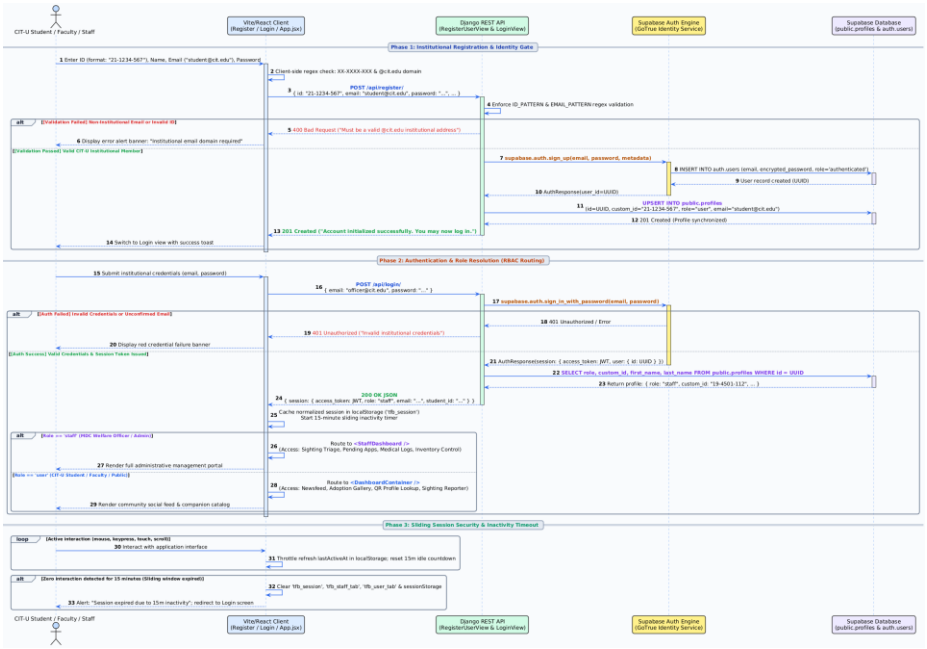
- **Use Case Description**

Use Case ID	UC-4.2
Use Case Name	Role-Based Access Control & Institutional Authentication
Primary Actor	CIT-U Student, Faculty, MDC Staff, Shelter Staff, Administrator
Preconditions	The user is attempting to register or log in to the portal.
Main Success Scenario	1. The User accesses the Login / Registration screen. 2. For registration, the System verifies that the Institutional ID matches regex format XX-XXXX-XXX (e.g., 21-1234-567) and the email ends with @cit.edu. 3. Upon valid submission, the backend hashes passwords via PBKDF2/bcrypt and creates the account in Supabase PostgreSQL. 4. Upon login, the backend authenticates credentials and issues a cryptographically signed JWT token pair. 5. The frontend inspects role claims and routes the user: - CLINIC or STAFF roles route to <StaffDashboard /> with medical logs and intake controls. - COMMUNITY role routes to <DashboardContainer /> with personal adoption applications. 6. The frontend starts a 15-minute sliding inactivity watchdog. 7. Active user interactions (keyboard, mouse, touch) continuously reset the 15-minute timer.
Alternative Flows	2a. Non-Institutional Email Entered: 1. User enters a non-CIT-U email address (e.g., @gmail.com or @yahoo.com). 2. The System rejects registration: 'Only official CIT-U email addresses (@cit.edu) are permitted.' 6a. 15 Minutes of Inactivity Elapsed: 1. Inactivity timer reaches zero without user input. 2. The System displays a session timeout notice, purges stored JWT tokens, and redirects to the login screen to protect shared campus computers.
Postconditions	Users access only their authorized modules, and abandoned sessions are securely terminated.

- **Activity Diagram**



- **Wireframe**


CIT-U Student / Faculty / Staff
Vite/React Client (Register / Login / App.jsx)
Django REST API (RegisterUserView & LoginView)
Supabase Auth Engine (GoTrue Identity Service)
Supabase Database (public.profiles & auth.users)
Phase 1: Institutional Registration & Identity Gate
1 Enter ID (format: "21-1234-567"), Name, Email ("student@cit.edu"), Password
2 Client-side regex check: XX-XXXX-XXX & @cit.edu domain
POST /api/register/
3 { id: "21-1234-567", email: "student@cit.edu", password: "...", ... }
4 Enforce ID_PATTERN & EMAIL_PATTERN regex validation
alt
[Validation Failed] Non-institutional Email or Invalid ID
5 400 Bad Request ("Must be a valid @cit.edu institutional address")
6 Display error alert banner: "Institutional email domain required"
[Validation Passed] Valid CIT-U Institutional Member
7 supabase.auth.sign_up(email, password, metadata)
8 INSERT INTO auth.users (email, encrypted_password, role='authenticated')
9 User record created (UUID)
10 AuthResponse(user_id=UUID)
11 UPSERT INTO public.profiles
{id=UUID, custom_id="21-1234-567", role="user", email="student@cit.edu"}
12 201 Created (Profile synchronized)
13 201 Created ("Account initialized successfully. You may now log in.")
14 Switch to Login view with success toast
Phase 2: Authentication & Role Resolution (RBAC Routing)
15 Submit institutional credentials (email, password)
POST /api/login/
16 { email: "officer@cit.edu", password: "..." }
17 supabase.auth.sign_in_with_password(email, password)
alt
[Auth Failed] Invalid Credentials or Unconfirmed Email
18 401 Unauthorized / Error
19 401 Unauthorized ("Invalid institutional credentials")
20 Display red credential failure banner
[Auth Success] Valid Credentials & Session Token Issued
21 AuthResponse(session: { access_token: JWT, user: { id: UUID } })
22 SELECT role, custom_id, first_name, last_name FROM public.profiles WHERE id = UUID
23 Return profile: { role: "staff", custom_id: "19-4501-112", ... }
200 OK JSON
24 { session: { access_token: JWT, role: "staff", email: "...", student_id: "..." } }
25 Cache normalized session in localStorage ('lfb_session')
Start 15-minute sliding inactivity timer
alt
[Role == 'staff' (NSTP Welfare Officer / Admin)]
26 Route to <StaffDashboard /> (Access: Sighting Triage, Pending Apps, Medical Logs, Inventory Control)
27 Render full administrative management portal
[Role == 'user' (CIT-U Student / Faculty / Public)]
28 Route to <DashboardContainer /> (Access: Newsfeed, Adoption Gallery, QR Profile Lookup, Sighting Reporter)
29 Render community social feed & companion catalog
Phase 3: Sliding Session Security & Inactivity Timeout
loop
[Active interaction (mouse, keypress, touch, scroll)]
30 Interact with application interface
31 Throttle refresh lastActiveAt in localStorage; reset 15m idle countdown
alt
[Zero interaction detected for 15 minutes (Sliding window expired)]
32 Clear 'lfb_session', 'lfb_staff_tab', 'lfb_user_tab' & sessionStorage
33 Alert: "Session expired due to 15m inactivity"; redirect to Login screen
CIT-U Student / Faculty / Staff
Vite/React Client (Register / Login / App.jsx)
Django REST API (RegisterUserView & LoginView)
Supabase Auth Engine (GoTrue Identity Service)
Supabase Database (public.profiles & auth.users)

3.4 Non-functional requirements

Performance

- **• Identification Speed:** The system must retrieve and display a pet's digital Public Pet Passport from a scanned QR code in under 1.5 seconds under standard campus Wi-Fi/4G conditions, and under 20 seconds under edge/low-bandwidth conditions.
- **• Reporting Latency:** Community sighting reports must be published and visible on the newsfeed within 3 seconds of submission.
- **• Concurrency:** The system must support at least 50 simultaneous users scanning codes, browsing pet listings, or viewing the newsfeed during peak campus hours without noticeable degradation.
- **• Cold-Start Keep-Alive:** An automated keep-alive cron job pinging GET /api/health/ every 10 minutes keeps the Render container awake, maintaining active API response latencies below 300ms.
- **• Tag Generation Speed:** The HTML5 canvas QR collar tag generator shall render a 300 DPI collar tag graphic in under 500ms client-side.

Security

- **• Access Control:** Strict role-based permissions must be enforced at the API layer. Only authorized MDC clinic staff can create or modify medical treatments, vaccination logs, and bite incident records.
- **• Institutional Domain Restriction:** User account registration is restricted exclusively to authorized CIT-U institutional email addresses (@cit.edu) and student/faculty ID format (XX-XXXX-XXX).
- **• Password Hashing:** All user passwords must be hashed using PBKDF2 with SHA-256 or bcrypt prior to database storage. Plaintext passwords are never stored or transmitted.
- **• Session Management:** User sessions must automatically terminate after 15 minutes of inactivity on client devices, purging stored authentication tokens to safeguard public campus terminals.
- **• Transport Encryption:** All web traffic and database transactions must be encrypted using HTTPS / TLS 1.3 to ensure data privacy.

Reliability

- **• Operational Uptime:** The system must maintain 99.5% availability during university operating hours (7:00 AM – 9:00 PM).
- **• Data Backup:** Automated daily backups of the PostgreSQL database and uploaded media assets are managed through Supabase backup schedules.
- **• Error Handling:** The system must provide clear, user-friendly error messages if a scan fails, if network connectivity drops, or if an invalid QR code is scanned.

Usability

- • **Usability Benchmark:** Addressing the baseline System Usability Scale (SUS) benchmark of 62.32 from MVP validation testing, the portal incorporates Module 4.1 (interactive user guide modal) to target an elevated SUS score > 80.0.
- • **Responsive Design:** The web interface must be fully responsive, with touch-friendly navigation optimized for standard mobile screens (375px–420px) and accessible desktop layouts.
- • **Accessibility Standards:** Visual status chips (Green for vaccinated, Red for bite quarantine, Amber for pending) must satisfy WCAG 2.1 AA color contrast standards.